Analysis of CTCP coding delay using ns3 modular transport

Anonymous Author(s)

ABSTRACT

We implement the network protocol coded TCP (CTCP) [4] in the ns3 simulator and analyze the impact of delay induced by coding operations on a variety of different network conditions. We find that, even with a naive implementation of coding operations, the induced delay has very little impact on the performance of CTCP. We show that CTCP outperforms TCP in a variety of lossy network conditions.

1 INTRODUCTION

Wireless networks suffer from a form of error not seen in wired networks, interference. Interference is a cause of random packet losses, and is one of the primary factors affecting network performance. In such networks, existing network protocols such as TCP have poor performance, and a great deal of research has been done into improving existing network protocols in lossy conditions [1]. A related line of research studies new network protocols for wireless networks, rather than trying to improve old ones. One such protocol is coded TCP.

Coded TCP (CTCP) was introduced by Kim et. al. in 2013 [4], building on work by Kumar et. al. [7], which uses concepts from network coding theory (see [2]) to make TCP more robust to lossy links. In CTCP, bytes are grouped into fixed size packets, and packets are grouped into blocks. The CTCP sender maintains a sliding window of blocks, and sends to the receiver packets which are random linear combinations of a blocks content. The CTCP receiver acknowledges the number of linearly independent combinations it has received so far, and when it has received enough to entirely decode a block, it transmits the decoded data to the application. To see why this would perform better in lossy network environments, imagine sending 10 packets over a link with 10% loss rate, and suppose that 1 packet is lost. In TCP, this packet will need to be retransmitted, and it may cause a triple duplicate ACK event, which will reduce the congestion window to half. In CTCP, the sender will send a slightly redundant number of packets, say 11, which are all coded linear combinations. With high probability over choice of coding coefficients, the receiver will receive 10 linearly independent combinations and decode the block without any need for retransmission.

CTCP has been tested both in simulations and in real-world experiments in [4]. The results found that CTCP consistently outperforms TCP when there is some random loss. In particular, simulations found that CTCP utilizes the link more efficiently than TCP when loss rates are as small as 0.1%, even when buffer sizes are small. In a real-world experiment comparing completion time for downloading a 50MB file over a busy public wifi network also shows that CTCP outperforms TCP by 100%-300%.

In the real-world experiments, the mean loss rate is between 4.28% - 5.25%, and the mean RTT is between 50.39ms to 106.31ms. The authors conclude that CTCP will outperform TCP on real wireless networks, and push for its adoption. Studies on real 5G networks have shown that, when packet sizes are 1024B, packet loss rates are as low as 2% [5]. Studies have also found that when signal strength is high in wireless networks, loss rates can be as low as 1% [6]. A major gap in the literature is the real-world performance of CTCP in environments where loss rates are below 2%. The simulation experiments of [4] ignore coding delay, as they expect it to have marginal impact on throughput.

1.1 Our Results

We confirm the intuition of [4] that coding delay of the CTCP receiver has a negligible impact on throughput. Even with a naive implementation of coding operations (we expect there is possibility for at least a 5x speed-up, as well as parallelization), almost all coding operations take less than 0.3ms, with the average being approximately 0.02ms. We also reproduce simulations showing that CTCP achieves significantly higher throughput than TCP even on links with relatively low loss rate. We do not assess the impact caused by sender-side coding delay, although we expect it to be very short as sender-side coding delay is incurred entirely by one matrix-vector product, which can be done very efficiently using specialized linear algebra libraries (i.e. LAPACK).

1.2 Organization

In section 2, we will give an overview of the CTCP sender operations and their implementations. In section 3, we will review the CTCP receiver operations and discuss the implementation of coding operations for our simulation. In section 4, we will discuss our experimental setup and results.

2 CTCP SENDER

2.1 Sending packets

The CTCP sender sends packets which are random linear combinations of block contents. In particular, each block is comprised of `blksize` strings of `SMSS` bytes. The CTCP sender views the bytes as vectors over $GF(256)$, the field of 256 elements, and sends a random linear combination of these vectors as a packet. Each packet has in its header a value for `blknum`, `seqno`, `seed`. `blknum` is the block whose information this packet carries. `seqno` is an *unique* sequence number given to each packet. We note that each sequence number is used *at most once* in a connection, and that retransmissions use different sequence numbers. Finally, the `seed` is used to seed a random number generator, from which the coding coefficients of the packet can be derived. In our implementation, `seed` is generated using `std::mt19937`, and the seed is used to seed the `srand` function.

The sender also maintains some information for each packet that is infilght. For each inflight packet, the sender stores the time it was sent, and the block it belongs to. This is used to estimate network parameters upon receiving an ACK. The sender uses the information in the ACK and the stored information to keep a short term estimate of the loss rate, and the RTT. The sender preemptively sends some redundant coded packets, so that the receiver will, in expectation, be able to decode all the blocks which have packets inflight.

2.2 Congestion Control

The CTCP sender keeps track of `tokens`, which represent the maximum number of inflight packets in the system. The sender also keeps track of a sliding window of blocks, from `currblk` to `currblk + numblks`, as well as the size of the blocks, `blksize`. These roughly correspond to the size of the receive window in TCP, and must be agreed upon in connection establishment, as it will affect the memory consumption of both the sender and the receiver. In our implementation, we use fixed values of both `blksize = 16` and `numblks = 100`.

The CTCP sender starts out in a slow-start phase, similar to TCP, and exits slow-start when the number of tokens exceeds `ssthresh`. In congestion-avoidance phase, the CTCP sender treats out-of-order ACKs as signals of loss. When an out-of-order ACK is received, the CTCP sender reduces the number of tokens available to it by `RTT / RTT_min`. If there is no congestion on the path, then this should not decrease the tokens available at all. If the CTCP sender does not receive a new ACK for a long period of time, then it resets its tokens to a small fixed number, and enters slow start phase.

Note that the standard TCP congestion control does not perform well on lossy links. Random loss causes TCP to decrease its congestion window to half, although there may not be any congestion on the link. This is due to an unsatisfied assumption in TCP's design, that loss is an indicator of congestion. This assumption is not satisfied on wireless links, where interfence causes random packet losses. Purely delay based congestion control, such as in TCP Vegas, suffers from very poor fairness when competing with standard TCP flows [3]. CTCP uses a combination of delay-based and loss-based signals to optimize congestion control over lossy links while not getting overshadowed by more aggressive protocols [4].

2.3 Coding operations

The CTCP sender performs two operations that may cause delay. First, the sender must generate random coefficients. This operation can be done extremely quickly, and takes a negligible amount of time. The second is to form the random linear combination of the payload. This operation is equivalent to computing one matrix-vector product between a matrix of size (SMSS, blksize) and a vector of length blksize. This can be done very efficiently using specialized libraries (i.e. LAPACK), and so we do not consider this delay.

3 CTCP RECEIVER

3.1 Acknowledgments

When the CTCP receiver gets a packet from the network, it adds it to a matrix corresponding to its block number, and performs partial Gaussian elimination. At this point, the receiver is able to tell if the newly received packet carried new information (its coefficients are linearly independent from the already received packets) or not. The receiver sends an ACK which acknowledges the sequence number of the received packet, the block number of the earliest block which is not decoded, and the number of degrees of freedom (dof) of that block. When a receiver has obtained `blksize` dof for a block, it decodes the packets by solving an upper triangular matrix, and then forwards the decoded bytes to the application.

3.2 Coding operations

The CTCP receiver has 3 coding operations it must do. First, it must use the seed in the packet header to generate the coding coefficients. Second, it must do a partial Gaussian elimination for each received packet. Third, when a block is ready to be decoded, it must solve an upper triangular matrix. All coding operations are done over $GF(256)$, which has a one-to-one correspondence between elements and bytes. We briefly discuss some implementation details of our Gaussian elimination and decoding algorithms. We note that arithmetic operations over $GF(256)$ are simple to implement; addition and subtraction correspond to XOR of the two byte strings, multiplication is done via a lookup into a precomputed multiplication table, and inverses are done via lookup

to a precomputed inverse table. The total memory footprint of these lookup tables is 65.7kB. In a low memory environment, an implementation can forgo an explicit multiplication table and instead compute log and exp tables, which would have a memory footprint of 512B, at the expense of (approximately) a factor of 3 slowdown.

We have chosen to implement the coding and payload matrices using `std::vector` as flat vectors in row major order. Pseudocode algorithms for our online Gaussian elimination and upper triangular solver are given in Algorithms 1 and 2. We note that storing matrices in *column* major order would make Algorithm 2 significantly more cache-friendly, however, Algorithm 1 runs once for every received packet, while Algorithm 2 only runs, on average, once every `blksize` packets. We also note that these is significant room for improvement in these algorithms. Most importantly, SIMD instructions can be used for both, and Algorithm 2 can easily be made parallel.

An optimization given in [4] that we have implemented is that the first `blkssize` packets sent for the current block will be uncoded, i.e. will correspond to the first `blksize` vectors constituting the block. In low loss environments, this greatly decreases the cost of both Gaussian elimination, and solving the upper triangular matrix. At the cost of slightly more memory, it is plausible to do this for every block, rather than just the current block, resulting in even faster coding operations.

Algorithm 1 Online Gaussian Elimination

Require: Matric C, Matrix P, vector c, vector p
1: index ⟵ first non-zero index of c
2: **while** index < blksize **do**
3: **if** index == C.rank() **then**
4: $c \leftarrow c/c[index]$, $p \leftarrow p/c[index]$
5: C[index] ⟵ c, P[index] ⟵ p
6: **return** true
7: **else**
8: c ⟵ c - C[index] / c[index]
9: p ⟵ p - P[index] / c[index]
10: index ⟵ first non-zero index of c
11: **end if**
12: **end while**
13: **return** false

Algorithm 2 Upper Triangular solver

Require: Matric C, Matrix P
1: **for** i = blksize-1,...,0 **do**
2: **for** j = 0,...,i-1 **do**
3: C[j] = C[j] - C[j][i] · C[i]
4: P[j] = P[j] - C[j][i] · P[i]
5: **end for**
6: **end for**

4 EXPERIMENTAL MEASUREMENTS

4.1 Experimental testbed

We compare our implementation of CTCP against the implementation of TCP in the ns3 modular transport library (code available https://git.uwaterloo.ca/mtahmasb/mtp-ns3/. We simulate a two node system with one sender and one receiver. We vary link parameters and packet error rates throughout our comparison, and measure induced coding delay using the built-in C++ timer, since the simulator does not account for coding delay on its own. All experiments show the transfer of a 5MB file over the link. Due to a bug in the ns3 simulator, we cannot simulate a drop tail queue in the link, so the queue is an infinite sized buffer. Due to this, we cannot accurately compare CTCP and TCP when the loss rate is 0.00; since no packets are ever dropped, TCP does not exist slow start stage and finishes very quickly. In this setting of 0 packet loss, CTCP should perform the same as TCP, as it will saturate the queue resulting in packet loss, which appears as out-of-order ACK, and then it will throttle its sending rate to 1/2 (if the queue has capacity of BDP as is standard).

4.2 Results

In figure 1, we plot the throughput and RTT of a CTCP transmission over a link with 0.5% packet drop rate. We emphasize that there is no fixed size buffer in this simulation due to a bug in the ns3 simulator, and so this situation is a good model of 5G networks, which are known to have extremely large buffers. In all cases, CTCP performs quite well, reaching near optimal throughput. In the third experiment, the effect of infinite sized buffers can be seen. CTCP quickly floods the buffer at the start of its connection, then experiences a time out event, sends a few packets, and waits a long time before receiving any feedback because of the excessive queuing time. This manifests in the RTT graph as a long period of extremely high RTT, and in the throughput graph as an extremely sharp drop from maximum throughput to almost 0 throughput. We note that the TCP implementation has a bug, resulting in transmission taking over 1000s, and hence has been omitted in this experiment.

Figure 2 shows a comparison of throughoutput between TCP and CTCP with a packet loss rate of 1.5% on a 1Mb/s link with 5ms delay. CTCP completes the transmission 4x faster than TCP, with an average throughput of 0.955 Mb/s compared to 0.247 Mb/s. I suspect this is another bug in the ns3 modular transport TCP implementation, or an artifact of the lack of a tail drop queue.

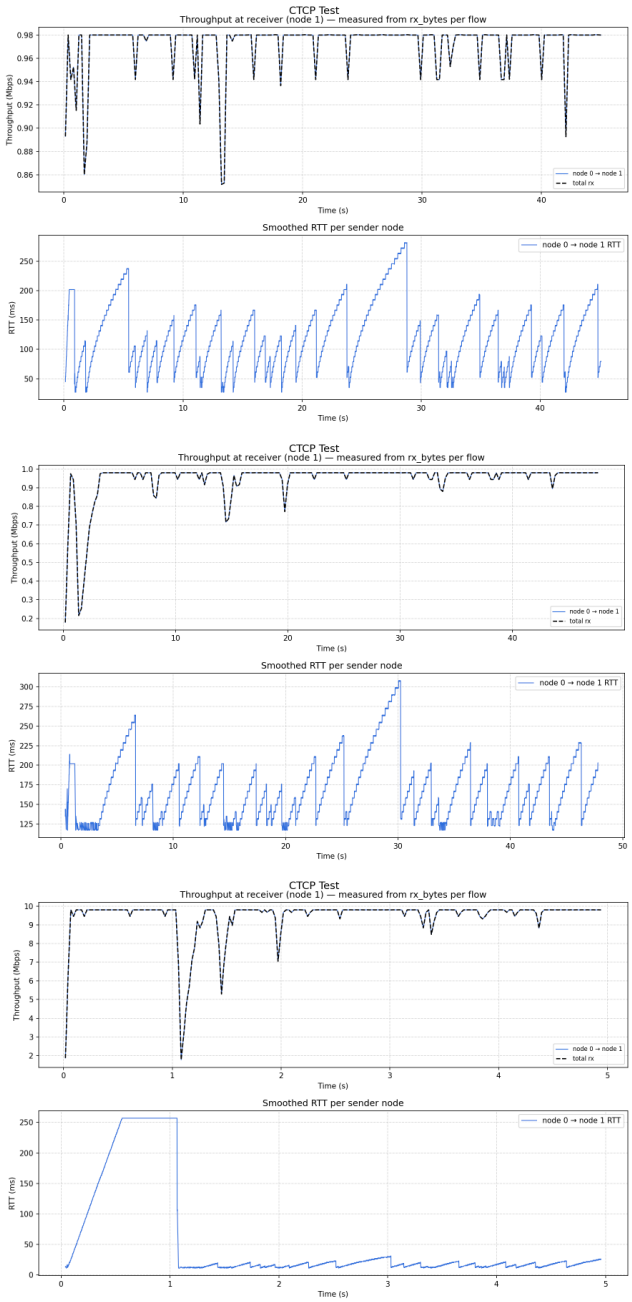


Figure 1: RTT and throughput of CTCP with 0.5% packet drop rate on a 1Mb/s bandwidth 5ms delay link (top), on a 1Mb/s bandwidth 50ms delay link (middle) and on a 10Mb/s bandwidth 5ms delay link (bottom). Time spent on coding operations totalled 616ms, 650ms, and 654ms respectively

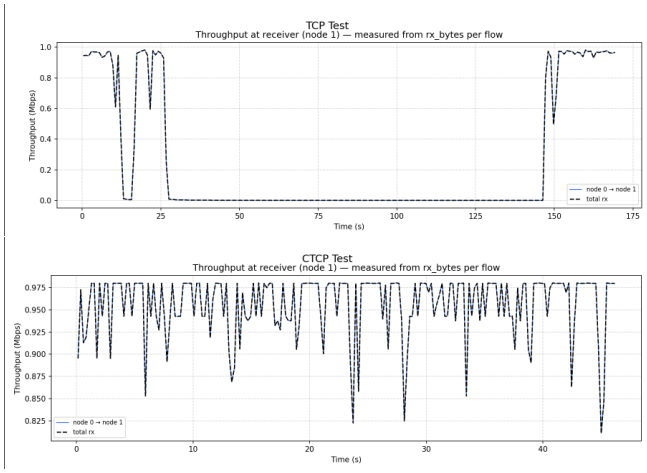


Figure 2: Throughout comparison of TCP (top) and CTCP (bottom) with 1.5% packet drop rate on a 1Mb/s bandwidth 5ms delay link. CTCP spent 545ms on coding operations.

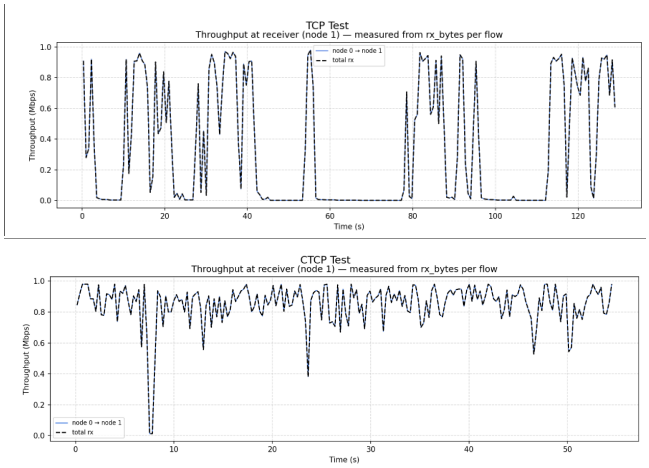


Figure 3: Throughout comparison of TCP (top) and CTCP (bottom) with 5.0% packet drop rate on a 1Mb/s bandwidth 5ms delay link. CTCP spent 419ms on coding operations.

Figure 3 compares throughput of CTCP and TCP under very lossy conditions. Here, CTCP has over a 100% improvement compared to TCP. We note that if there were a properly provisioned queue, both protocols would achieve better performance. TCP especially will benefit from a large queue, as currently it overloads the buffer resulting in bufferbloat, and then when must wait almost an RTO before sending more data. Nonetheless, this may be the condition in a wireless network.

5 DISCUSSION

In all of our experiments, we have found that CTCP outperforms TCP in terms of throughput. Moreover, we have found that the coding delay incurred by CTCP is two orders of magnitude less than the time of the transmission. The average time consumed for a Gaussian elimination step is 0.0025ms, and the average time consumed to decode a block is 0.45ms. Observe that an ACK can be sent before decoding a block, and so decoding of the block will not increase delay from the perspective of the sender. From the perspective of the application, decoding delay will increase perceived latency, so latency sensitive applications will suffer. However, CTCP is already a poor choice for such applications, and data is only transmitted to the application every `blksize` or so packets. We note that the decoding step can also be parellelized, and we estimate SIMD operations can increase the speed by a factor of 5. Overall, unless loss rates are very low and the application is very latency sensitive or has limited compute, CTCP seems to be a better choice than TCP. This is the case even if the link has a very moderate loss rate of 1.5%.

We do note that the results of these experiments must be interpreted carefully, as the ns3 simulator does not allow for drop tail queues. This means that the simulations are a good model for 5G or mobile networks, where queues are very large and TCP is susceptible to bufferbloat.

6 SUMMARY AND FUTURE WORK

In this survey we analyzed the effect of coding delay in CTCP, and found that even on links with relatively low loss rate, CTCP outperforms TCP despite incurring some small coding delay. We found that the most important coding delay, related to Gaussian elimination, takes an average of 0.0025ms per packet. This is a negligible delay in all but the most latency sensitive application, for which CTCP is not a suitable choice. We note that coding delay can be reduced even further by a more sophisticated implementation which makes better use of harware support for parallelization.

It remains to be studied how CTCP performs against other more competitive protocols in a wireless setting, such as Sprout [8]. In these settings, could the coding delay have a larger impact? It also remains to be seen what the optimal setting of `blksize` is. Clearly, increasing `blksize` will increase coding delay, but it will also decrease the frequency of decoding blocks. When the application is not sensitive to slight delays in receiving data from the network, this could further reduce the delay induced by the Gaussian elimination step. It also remains to study how CTCP performs and compares to TCP in the presence of a queue provisioned with a buffer of size BDP, as is standard. This is expected to increase the performance of both CTCP and TCP, but disproportionately improve the performance of TCP. Lastly, it would be interesting to see if the memory overhead required to make CTCP efficient can be reduced. This would make it much more suitable for embedded systems or machines with very limited resources.

REFERENCES

[1] B.S. Bakshi, P. Krishna, N.H. Vaidya, and D.K. Pradhan. 1997. Improving performance of TCP over wireless networks. In *Proceedings of 17th International Conference on Distributed Computing Systems*. 365–373. https://doi.org/10.1109/ICDCS.1997.598070

[2] Christina Fragouli, Jean-Yves Le Boudec, and Jörg Widmer. 2006. Network coding: an instant primer. *SIGCOMM Comput. Commun. Rev.* 36, 1 (Jan. 2006), 63–68. https://doi.org/10.1145/1111322.1111337

[3] U. Hengartner, J. Bolliger, and T. Gross. 2000. TCP Vegas revisited. In *Proceedings IEEE INFOCOM 2000. Conference on Computer Communications. Nineteenth Annual Joint Conference of the IEEE Computer and Communications Societies (Cat. No.00CH37064)*, Vol. 3. 1546–1555 vol.3. https://doi.org/10.1109/INFCOM.2000.832553

[4] MinJi Kim, Jason Cloud, Ali ParandehGheibi, Leonardo Urbina, Kerim Fouli, Douglas Leith, and Muriel Medard. 2013. Network Coded TCP (CTCP). arXiv:1212.2291 [cs.NI] https://arxiv.org/abs/1212.2291

[5] Justus Rischke, Peter Sossalla, Sebastian Itting, Frank H. P. Fitzek, and Martin Reisslein. 2021. 5G Campus Networks: A First Measurement Study. *IEEE Access* 9 (2021), 121786–121803. https://doi.org/10.1109/ACCESS.2021.3108423

[6] Ramanujan K Sheshadri and Dimitrios Koutsonikolas. 2017. On packet loss rates in modern 802.11 networks. In *IEEE INFOCOM 2017 - IEEE Conference on Computer Communications*. 1–9. https://doi.org/10.1109/INFOCOM.2017.8057130

[7] Jay Kumar Sundararajan, Devavrat Shah, Muriel Médard, Szymon Jakubczak, Michael Mitzenmacher, and João Barros. 2011. Network Coding Meets TCP: Theory and Implementation. *Proc. IEEE* 99, 3 (2011), 490–512. https://doi.org/10.1109/JPROC.2010.2093850

[8] Keith Winstein, Anirudh Sivaraman, and Hari Balakrishnan. 2013. Stochastic forecasts achieve high throughput and low delay over cellular networks. In *Proceedings of the 10th USENIX Conference on Networked Systems Design and Implementation* (Lombard, IL) *(nsdi'13)*. USENIX Association, USA, 459–472.